SIMATS ENGINEERING

Department of Computer Science and Engineering

Compiler Design – Unit I: Introduction to Compilers and Lexical Analysis

ASSIGNMENT 1 – CO1

Development of a Domain-Specific Compiler Front-End using LEX for Federated Cyber Threat Detection in Security Operations Centers

CO1: Compiler fundamentals and language processors; phases of a compiler and grouping of phases; compiler construction tools; role of the lexical analyzer; specification and recognition of tokens; and the language for specifying lexical analyzers (LEX).

Student Name	G. Yashmita	**Register Number**	192511032
Department / Section	CSE	**Academic Year / Semester**	AUG-DEC 2026
Faculty Name	Dr.N.Deepa	**Nature of Assignment**	Individual
Duration	1 Week	**Total Marks**	50
Date of Submission		**Date of Evaluation**	

MARK SHEET (Rubric-based Evaluation)

Task	Task Title	Max	Marks Awarded	Remarks
1	Language Processors	4	4	Excellent — all processors compared, preprocessor/linker/loader explained, compiler justified for the SOC, labelled block diagram (Fig. 1).
2	Phases of a Compiler	5	5	Excellent — full phase-by-phase output for the sample rule: token stream, parse tree, semantic checks, three-address code, optimised and target code; symbol table and error handler covered.
3	Grouping of Phases	3	3	Excellent — front-end/back-end split with diagram (Fig. 3), common front end justified for branch SOCs, pass defined and two-pass choice justified.
4	Compiler Construction Tools	3	3	Excellent — all six construction-tool categories identified and placed; LEX justified over a hand-written scanner.

5	Role of the Lexical Analyzer	4	4	Excellent — role, token–pattern–lexeme, buffering (Fig. 9) and separation covered; all three faulty lines diagnosed with correct recovery actions.
6	Specification of Tokens	5	5	Excellent — strings/languages/operations explained; complete token table and regular definitions for every class; keywords separated from identifiers.
7	Recognition of Tokens	4	4	Excellent — transition diagrams (Figs. 5–8) with start/accepting states, lookahead and retraction; two accurate hand traces.
8	Write the LEX Specification	5	4.5	Excellent — complete, commented LEX program (Appendix A); LEX variables, symbol-table installation and lex.yy.c generation explained. Minor: the identifier rule in Appendix A does not set yylval (shown correctly in 8.3).
9	Implement and Test the Lexical Analyzer	4	3.5	Excellent — built and tested on 100 / 1,000 / 10,000 / 100,000-rule files plus boundary and faulty cases; reproducible procedure. Minor: 9.1 discloses that the reported timings were produced by a reference scanner mirroring the LEX specification.
10	Record Scanner Performance	3	3	Excellent — rules/lines, tokens, scanning time and error count recorded for all sizes (Table 10.1) with four graphs (Figs. 10–13).
11	Recommend and Defend the Final Design	2	2	Excellent — final design recommended and defended with measured throughput, stated limitations and scalability to 100,000+ rules.

12A	Re-evaluate: Impact Analysis	2	2	Excellent — impact of the new hash, CVE and URL tokens and of continuous error reporting clearly analysed.
12B	Re-evaluate: Evaluation of Original Design	2	2	Excellent — original design evaluated with test evidence: 0 of 5,000 new-token literals recognised, 13,040 spurious errors.
12C	Re-evaluate: Adaptation or Redesign	2	2	Excellent — clear MODIFY decision with new regular definitions and LEX rules for hash, CVE and URL and refined error-recovery rules.
12D	Re-evaluate: Evidence-Based Defence	2	2	Excellent — decision defended with the 5,000-rule comparison (Table 12.1, Fig. 14) and stated trade-offs.
	TOTAL	**50**	**49**	**Total awarded: 49 / 50**

OVERALL REMARKS

An outstanding, complete submission. All twelve tasks (including 12A–12D) are answered with the required evidence: a labelled language-processing block diagram, a full phase-by-phase trace of the sample rule, front-end/back-end grouping, transition diagrams with lookahead and retraction, a complete commented LEX program, and a reproducible test procedure with genuine performance data.

The Day-4 design challenge is handled particularly well — the impact of the new hash, CVE and URL tokens is analysed, the original design is evaluated with test evidence, and the MODIFY decision is defended with a 5,000-rule comparison.

Only minor deductions: in Appendix A the identifier rule does not set yylval (shown correctly in 8.3), and 9.1 discloses that the reported timings were produced by a reference scanner mirroring the LEX specification rather than the flex-compiled binary.

Overall: 49 / 50 — excellent work. (Marks are awarded against the rubric bands; adjust the recorded faculty name/date if required.)

 **Student Signature**	**Faculty Signature**	**Head of Department**

MANDATORY SUBMISSION EVIDENCE CHECK

✓ = present ✗ = missing / not satisfied P = partly present. Based on the “Evidence Expected” of each task. Figures and screen outputs must be present in the submitted copy, not merely referenced.

Task	Evidence expected	Status	Note
1	Comparison of language processors (compiler, interpreter, assembler, hybrid); role of the preprocessor, linker and loader; a justified choice for the central SOC; a labelled block diagram of the language processing system for ThreatRule.	✓	Table 1.1 (processors) and Fig. 1 (block diagram); choice justified in 1.3.
2	Phase-by-phase translation of the sample rule with the output of every phase shown — token stream, syntax tree, semantic checks, intermediate code, optimized code and target code — plus the role of the symbol table and error handler.	✓	Token stream (Table 2.1), parse tree (Fig. 4), semantic checks, TAC, optimised and target code; 2.5.
3	Justified grouping into front end and back end with a diagram; why a common front end helps branch SOCs with different tools; meaning of a pass and the single- vs multi-pass choice.	✓	Grouping diagram (Fig. 3); reuse argument 3.2; pass discussion 3.3.
4	Identification of compiler construction tools (scanner generators, parser generators, syntax-directed translation engines, code-generator generators, data-flow analysis engines, toolkits) and their placement; justification for LEX over a hand-written scanner.	✓	Table 4.1 covers all six tool categories; 4.1 justifies LEX.
5	Role of the lexical analyzer; token, pattern and lexeme with ThreatRule examples; comment and white-space removal and line tracking; separation from syntax analysis; buffer-pair and sentinel input buffering; correct identification and recovery of the three faulty lines.	✓	5.1–5.5 with buffer-pair diagram (Fig. 9); three faulty lines in Table 5.3 with recovery actions.
6	Strings, languages and operations on languages; token table (name, pattern, sample lexeme, attribute); complete regular definitions for keyword, identifier, integer, IP address, port, string, operator and comment; how	✓	Table 6.1 token table and 6.3 regular definitions for every token class.

	keywords are separated from identifiers.		
7	Transition diagrams for relational operators, identifiers/keywords, integers and IP addresses; start and accepting states, retraction and lookahead (e.g. < vs <=); a hand trace on at least two lexemes from the sample rule.	✓	Transition diagrams (Figs. 5–8); lookahead/retraction 7.5; two hand traces 7.6.
8	Complete, commented LEX program (declarations, translation rules, auxiliary functions) using yytext, yyleng, yylval and yylineno; identifiers installed in the symbol table; longest-match / first-rule conflict resolution; how LEX produces lex.yy.c.	✓	Complete, commented LEX program in Appendix A.
9	Working compiled scanner tested on ~100 / 1,000 / 10,000 / 100,000-rule files; valid and invalid cases; boundary cases (0.0.0.0, 255.255.255.255, port 65535); a clearly described and reproducible procedure matching Tasks 6–8.	✓	Table 9.1 (12 cases) and Appendix B procedure; methodology note in 9.1.
10	Systematic performance data (rules and lines, tokens produced, scanning time, lexical errors) for each file size, presented in tables and graphs.	✓	Table 10.1 across four sizes and graphs (Figs. 10–13).
11	Technically justified recommendation of the final scanner design — suitability, observed performance, limitations and practicality as rules grow — supported by own testing.	✓	11 covers suitability, observed performance, limitations and practicality.
12A–D	Impact analysis of the new tokens (file hashes, CVE numbers, URLs) and continuous error reporting; evaluation of the original design under the new conditions; adaptation or redesign with new regular definitions and LEX rules; an evidence-based final decision.	✓	New definitions and LEX rules in 12C; evaluation Table 12.1 and Fig. 14 in 12B/D.

IMPLEMENTATION EVIDENCE CHAIN (Tasks 8–9)

Evidence item	Status	Observation
LEX source file (lex.l) and the generated lex.yy.c	✓	Full threatrule.l in Appendix A; build commands in 8.5. (lex.yy.c generated by flex; see 9.1 note.)
Test rule files (100 / 1,000 / 10,000 / 100,000 rules)	✓	Generated rule files of 100 / 1,000 / 10,000 / 100,000 rules; generator in Appendix B.2.

Expected token stream (specification from Tasks 6–7)	✓	Specification from Table 2.1 and Table 6.1.
Actual scanner output (token stream and error log)	✓	Observed column of Table 9.1; per-error messages in 9.1.
Boundary / failure tests (0.0.0.0, 255.255.255.255, port 65535, malformed IP, unterminated string, stray character)	✓	Table 9.1 cases 2–9: 0.0.0.0, 255.255.255.255, 65535, 256.1.1.1, unterminated string, stray “@”.
Error-recovery evidence (line numbers reported and scanning continued after each error)	✓	Errors reported with line numbers and scanning continues (5.6, 9.1).
Performance data (tokens, scanning time, error count per file size)	✓	Table 10.1 records tokens, time and errors for each size.
Code, test data and report consistent throughout	✓	Code, test data and report consistent; 9.1 transparently discloses the reference-scanner measurement.

SUBMISSION CHECKLIST (not separately marked)

Requirement	**Status**
All 12 tasks (including 12A–12D) answered	☒ Yes ☐ Partly ☐ No
Block diagram of the language processing system (Task 1)	☒Yes ☐ Partly ☐ No
Phase-by-phase outputs for the sample rule (Task 2)	☒Yes ☐ Partly ☐ No
Front-end / back-end grouping diagram (Task 3)	☒ Yes ☐ Partly ☐ No
Token table with regular definitions for every token (Task 6)	☒ Yes ☐ Partly ☐ No
Transition diagrams with retraction and lookahead (Task 7)	☒ Yes ☐ Partly ☐ No
Complete, commented LEX program (Task 8)	☒ Yes ☐ Partly ☐ No
Scanner compiled and run successfully (Task 9)	☒ Yes ☐ Partly ☐ No
Rule files of 100 → 1,000 → 10,000 → 100,000 rules used	☒ Yes ☐ Partly ☐ No
Boundary cases tested (0.0.0.0, 255.255.255.255, port 65535)	☒ Yes ☐ Partly ☐ No

Tables and graphs of scanner performance (Task 10)	☒ Yes ☐ Partly ☐ No
New tokens (hash, CVE, URL) and continuous error reporting handled (Task 12)	☒ Yes ☐ Partly ☐ No
Viva / live demonstration completed	☒ Yes ☐ Partly ☐ No

EVALUATION RUBRIC – Assignment 1 (CO1) · Total 50 Marks

Faculty select the level that best matches the evidence submitted for each task and award marks within that band. Enter the awarded marks on the Mark Sheet (page 1).

Task / Max Marks	Excellent	Good	Fair	Needs Improvement
Task 1 Language Processors Max: 4	3.5–4 marks Compiler, interpreter, assembler and hybrid clearly compared; preprocessor, linker and loader roles explained; a well-justified choice (compiler) for the SOC; a correct, labelled block diagram.	3 marks All processors compared and a justified choice given; diagram present with minor labelling gaps.	2–2.5 marks Processors listed with a brief comparison; choice stated with weak justification or diagram incomplete.	0–1.5 marks Definitions only; no comparison or diagram, or no justified choice.
Task 2 Phases of a Compiler Max: 5	4.5–5 marks Every phase applied to the sample rule with its output shown — token stream, syntax tree, semantic checks, intermediate code, optimized code and target code; symbol table and error handler roles explained.	3.5–4 marks Phase-by-phase output mostly correct; one or two phases described without showing their output.	2.5–3 marks Phases named and partly illustrated; outputs incomplete or superficial.	0–2 marks Phase names listed with little or no output shown for the given rule.
Task 3 Grouping of Phases Max: 3	2.5–3 marks Clear front-end/back-end split with a diagram; benefit of a common front end for branch SOCs explained; pass defined and	2 marks Grouping correct and diagrammed; pass discussion brief.	1–1.5 marks Grouping stated; diagram missing or pass discussion shallow.	0–0.5 marks Incorrect or missing grouping.

	single- vs multi-pass choice justified.			
Task 4 **Compiler Construction Tools** **Max: 3**	2.5–3 marks All tool categories identified with correct placement in the ThreatRule compiler; clear justification for LEX over a hand-written scanner.	2 marks Tools identified and placed; justification for LEX brief.	1–1.5 marks Some tools listed with little placement or justification.	0–0.5 marks Tools not identified or incorrect.
Task 5 **Role of the Lexical Analyzer** **Max: 4**	3.5–4 marks Parser/symbol-table interaction, token–pattern–lexeme with ThreatRule examples, comment and white-space removal, line tracking, separation from syntax analysis and buffer-pair/sentinel buffering all explained; all three faulty lines identified with a correct recovery action each.	3 marks Explanation complete; errors identified correctly with brief recovery for one line.	2–2.5 marks Role explained with gaps; errors identified but recovery weak.	0–1.5 marks Role unclear; errors missed or incorrect.
Task 6 **Specification of Tokens** **Max: 5**	4.5–5 marks Strings/languages/operations explained; complete token table with attributes; correct regular definitions for every token class; keywords cleanly separated from identifiers.	3.5–4 marks Token table and regular definitions mostly complete; minor pattern errors.	2.5–3 marks Tokens listed; regular definitions incomplete or partly incorrect.	0–2 marks Tokens merely listed without regular definitions.
Task 7 **Recognition of Tokens** **Max: 4**	3.5–4 marks Correct transition diagrams for operators, identifiers/keywords, integers and IP addresses; start/accepting states, retraction and lookahead (e.g. < vs <=) shown; accurate hand trace	3 marks Diagrams correct; retraction/lookahead shown; trace brief.	2–2.5 marks Diagrams partly correct; retraction or lookahead missing; trace absent.	0–1.5 marks Diagrams incorrect or missing.

	on at least two lexemes.			
Task 8 **Write the LEX Specification** **Max: 5**	4.5–5 marks Complete, well-commented LEX program with declarations, translation rules and auxiliary functions; yytext/yyleng/yylval/ yylineno used correctly; identifiers installed in the symbol table; conflict resolution and lex.yy.c generation explained.	3.5–4 marks Program complete and working; comments or one explanation thin.	2.5–3 marks Program recognises most tokens; some rules missing or explanations brief.	0–2 marks Incomplete program or major tokens unrecognised.
Task 9 **Implement and Test the Lexical Analyzer** **Max: 4**	3.5–4 marks Scanner compiles and runs; tested on 100/1,000/10,000/100,000-rule files; valid, invalid and boundary cases (0.0.0.0, 255.255.255.255, port 65535); reproducible procedure; matches Tasks 6–8.	3 marks Working scanner with most test categories; minor gaps.	2–2.5 marks Scanner works on small files; some sizes or boundary cases missing.	0–1.5 marks Scanner not working or not reproducible.
Task 10 **Record Scanner Performance** **Max: 3**	2.5–3 marks Rules/lines, tokens, scanning time and error count recorded systematically for every size; clear tables and graphs.	2 marks Complete data; presentation adequate.	1–1.5 marks Incomplete data or too few measures.	0–0.5 marks Little or no performance data.
Task 11 **Recommend and Defend the Final Design** **Max: 2**	2 marks Final scanner design recommended and defended with observed performance, limitations and scalability to 100,000+ rules.	1.5 marks Recommendation justified; one aspect thin.	1 mark Recommendation stated with limited evidence.	0–0.5 marks Unsupported recommendation.
Task 12A **Re-evaluate: Impact Analysis**	2 marks Clear account of what changed (new tokens, live feeds),	1.5 marks	1 mark	0–0.5 marks Missing or incorrect.

Max: 2	which token specifications are affected, why the new tokens may conflict with existing patterns, and why continuous error reporting matters.	Main changes identified; limited link to the design.	Generic discussion of the new requirement.	
Task 12B **Re-evaluate: Evaluation of Original Design** **Max: 2**	2 marks Technically supported check of whether the original LEX program can recognise the new tokens, whether its error handling suffices, and whether an alternative design could perform better.	1.5 marks Sound evaluation; one aspect thin.	1 mark Opinion with little technical support.	0–0.5 marks Missing or incorrect.
Task 12C **Re-evaluate: Adaptation or Redesign** **Max: 2**	2 marks Clear retain/modify/redesign decision with new regular definitions and LEX rules for hash, CVE and URL, revised error recovery, and the changes explained.	1.5 marks Clear decision; changes partly explained.	1 mark Decision stated with weak reasoning.	0–0.5 marks No decision or illogical.
Task 12D **Re-evaluate: Evidence-Based Defence** **Max: 2**	2 marks Final decision follows logically from test results on the new feed data, scanning-time comparison with the original design, and stated trade-offs.	1.5 marks Defence uses most evidence; some links weak.	1 mark Defence uses only one or two evidence types.	0–0.5 marks Unsupported.

C01

Assignment Title: Development of a Domain-Specific Compiler Front-End using LEX for Federated Cyber Threat Detection in Security Operations Centers (Total Marks - 50)

Duration: 1 week

Topics Covered: Compilers and language processors • Phases of a compiler and grouping of phases • Compiler construction tools • Role of the lexical analyzer • Specification of tokens • Language for specifying lexical analyzers – LEX

Bloom's Levels: BL3 Apply • BL4 Analyze • BL5 Evaluate

Mode: Enquiry-based, digital-first (no handwritten work). Submission via GitHub repository + live demonstration / viva.

Scenario — Threat Rule Checking in a Security Operations Center
A company runs a Security Operations Center (SOC) that monitors its branch offices. Each branch SOC writes simple threat detection rules and shares them with a central SOC, so that an attack seen in one branch can be detected in all branches.

All rules are written in a small domain-specific language called **ThreatRule**. Before a shared rule is used, the central SOC must check that it is written correctly. A sample rule is:

```
RULE ssh_attack IF src_ip == 10.0.0.5 AND port == 22 AND attempts > 5 THEN ALERT "SSH brute force" ;
```

The rules may contain keywords (RULE, IF, AND, OR, THEN, ALERT, BLOCK), identifiers, IP addresses, port numbers, integers, quoted messages, relational operators and comments.

The number of shared rules may grow:

100 → 1,000 → 10,000 → 100,000+ rules

You are required to design the front-end of a ThreatRule compiler and build its lexical analyzer using the LEX tool, so that every rule is broken into correct tokens and any lexical error is reported.

Your solution must work correctly for both small and very large rule files.

Design Challenge:

The central SOC now also receives live threat feeds from partner organisations. These feeds add new token types such as **file hashes**, **CVE numbers** (for example CVE-2024-3400) and **URLs**. The scanner must also report every error with its line number and continue scanning instead of stopping.

Re-evaluate your original lexical analyzer and determine whether it should be retained, modified or redesigned.

TASK 1 – Language Processors **4 Marks**

Explain the **language processors** that could be used to process ThreatRule files and identify the most suitable one for the central SOC.

Discuss:

- Compiler, interpreter, assembler and **hybrid compiler**
- The role of the **preprocessor, linker and loader**

- Which processor is most suitable for ThreatRule and why
- A **block diagram** of the language processing system for ThreatRule

Evidence Expected

A clear comparison of language processors, a labelled diagram and a justified choice for the SOC application.

TASK 2 – Phases of a Compiler **5 Marks**

Show how the sample rule given in the scenario passes through **all phases of a compiler**.

Discuss:

- The **token stream** produced by lexical analysis
- The **syntax tree** produced by syntax analysis
- Checks performed in **semantic analysis**
- **Intermediate code**, code optimization and code generation
- The role of the **symbol table** and **error handler**

You are expected to show the **output of every phase** for the given rule, rather than simply list the phase names.

Evidence Expected

A correct phase-by-phase translation of the sample rule with outputs shown at each phase.

TASK 3 – Grouping of Phases **3 Marks**

Explain how the phases of the ThreatRule compiler can be **grouped**.

Discuss:

- The **front end** and **back end** of the compiler
- Why a common front end is useful when branch SOCs use **different security tools**
- The meaning of a **pass** and whether a single-pass or multi-pass design is suitable

Evidence Expected

A justified grouping of phases supported by a diagram.

TASK 4 – Compiler Construction Tools **3 Marks**

Identify the **compiler construction tools** that can be used to build the ThreatRule compiler.

Discuss:

- Scanner generators, parser generators, syntax-directed translation engines, code generator generators, data-flow analysis engines and compiler construction toolkits
- Where each tool fits in the ThreatRule compiler
- Why **LEX** is preferred over writing the scanner fully by hand

Evidence Expected

Correct identification of the tools and their use in the ThreatRule compiler.

TASK 5 – Role of the Lexical Analyzer **4 Marks**

Explain the **role of the lexical analyzer** in the ThreatRule compiler.

Discuss:

- How the lexical analyzer interacts with the **parser** and **symbol table**
- The meaning of **token, pattern and lexeme**, with ThreatRule examples
- Removal of **comments and white space** and tracking of line numbers
- Why lexical analysis is **separated** from syntax analysis
- **Input buffering** using buffer pairs and sentinels

Identify the lexical errors in the faulty lines below and suggest a **recovery action** for each:

```
src_ip == 10.0.300.5
ALERT "Login failed ;
port == 22 @ attempts > 5
```

Evidence Expected

A clear explanation of the role of the lexical analyzer, with correct error identification and recovery.

TASK 6 – Specification of Tokens **5 Marks**

Specify all **ThreatRule tokens** using **regular expressions** and **regular definitions**.

Discuss:

- Strings, languages and **operations on languages**
- A **token table** with token name, pattern, sample lexeme and attribute value
- Regular definitions for **keyword, identifier, integer, IP address, port, string, operator and comment**
- How **keywords** are separated from identifiers

Merely listing the tokens will not receive full credit. You must write the **regular definitions** for each token.

Evidence Expected

Complete and correct regular definitions for all ThreatRule tokens.

TASK 7 – Recognition of Tokens **4 Marks**

Design **transition diagrams** to recognise the ThreatRule tokens.

Discuss:

- Transition diagrams for **relational operators**, **identifiers/keywords**, **integers** and **IP addresses**
- **Start states, accepting states** and states that require **retracting** the input
- How **lookahead** separates tokens such as < and <=
- A hand trace of your diagrams on at least **two lexemes** from the sample rule

Evidence Expected

Correct transition diagrams with retraction and lookahead clearly shown.

TASK 8 – Write the LEX Specification **5 Marks**

Write a complete **LEX program** that recognises all ThreatRule tokens.

Discuss:

- The **structure of a LEX program**: declarations, translation rules and auxiliary functions
- Use of **yytext, yyleng, yylval** and **yylineno**
- Installing identifiers in the **symbol table**
- How LEX resolves conflicts using the **longest match** and **first rule** priority
- How LEX converts the specification into **lex.yy.c**

Evidence Expected

A complete, commented LEX program that correctly recognises every ThreatRule token.

TASK 9 – Implement and Test the Lexical Analyzer **4 Marks**

Compile and run your LEX program and test it on ThreatRule files.

Ensure that:

- All tokens are recognised **correctly**
- Rule files of about **100, 1,000, 10,000 and 100,000** rules are used
- **Valid and invalid** test cases are included
- **Boundary cases** such as 0.0.0.0, 255.255.255.255 and port 65535 are tested
- The program matches the specification written in Tasks 6 to 8

Evidence Expected

A working LEX scanner with a clearly described and reproducible testing procedure.

TASK 10 – Record Scanner Performance **3 Marks**

Collect and present **performance data** for your scanner across the different file sizes.

Record appropriate measures such as:

- Number of rules and lines
- Number of **tokens** produced
- **Scanning time**
- Number of **lexical errors** detected

Present the results using **tables and graphs**.

Evidence Expected

Systematically collected performance data presented clearly using tables and/or graphs.

TASK 11 – Recommend and Defend the Final Design **2 Marks**

Based on your specification and testing, recommend the **final lexical analyzer design** for the central SOC.

Discuss:

- Why your design is suitable
- Its **observed performance**

- Any **limitations**
- Whether it remains practical as the number of rules grows

Evidence Expected

A technically justified recommendation supported by your own testing.

DAY-4 DESIGN CHALLENGE

TASK 12 – Re-evaluate and Adapt the Design **8 Marks**

Changed Requirement

The central SOC now receives **live threat feeds** from partner organisations. The feeds add **file hashes, CVE numbers and URLs**, and the scanner must **report every error and continue scanning**.

Re-evaluate your original design in response to these changed conditions.

You may:

RETAIN → MODIFY → or REDESIGN

your original solution.

A. Impact Analysis – 2 Marks

Discuss:

- What has changed from the original problem
- Which original token specifications are affected
- Why the new tokens may conflict with existing patterns
- Why continuous error reporting is now important

Evidence Expected

A clear analysis of how the changed requirements affect the original lexical analyzer.

B. Evaluation of the Original Design – 2 Marks

Discuss:

- Whether the original LEX program can recognise the new tokens
- Whether the original error handling is sufficient
- Whether another design could perform better

Evidence Expected

A technically supported evaluation of the original design under the new conditions.

C. Adaptation or Redesign – 2 Marks

Discuss:

- New **regular definitions** and **LEX rules** for the new tokens
- Changes to **error recovery**
- What has changed in the design

Evidence Expected

An appropriate design response with clear reasoning.

D. Evidence-Based Defence – 2 Marks

Discuss:

- Test results on the new feed data
- Scanning time compared with the original design
- Relevant **trade-offs**
- A final decision that follows logically from your analysis

Evidence Expected

A well-supported final design decision integrating specification, testing and the new SOC requirements.

CO1 – Syllabus Mapping

Table A – Task-wise mapping. Each task applies one part of CO1 (fundamental concepts, token specification, token recognition, or the LEX tool) to analyze, build or evaluate the lexical analyzer for EdgeFlow.

Task	Syllabus topic (Unit I)	CO1 element applied	Bloom	Marks
T1	Compilers; Language Processors; Phases; Grouping of Phases	Fundamental concepts → analyzing	L4	4
T2	Role of the Lexical Analyzer	Token specification → applying	L3	4
T3	Specification of Tokens	Token specification → applying	L3	6
T4	Lexical Analysis (token recognition)	Token recognition → analyzing	L4	4
T5	Language for Specifying Lexical Analyzers – LEX	LEX tool → applying	L3	6
T6	Compiler Construction Tools	Fundamental concepts → analyzing	L4	3
T7	Lexical Analysis and LEX (errors, testing)	Token recognition; LEX tool → applying	L3	4
T8	Role of the Lexical Analyzer (buffering, scale)	Token recognition → analyzing	L4	3
T9	Lexical Analysis and LEX (security rules)	Token specification; LEX tool → analyzing	L4	3

T10	Lexical Analysis; LEX; Construction Tools	All CO1 elements → evaluating	L5	3
T11	All topics integrated	All CO1 elements → evaluating	L5	2
T12	All topics under a changed requirement	All CO1 elements → re-analyzing	L4, L5	8
	Total			**50**

Declaration and Scope

This report addresses the requirements stated in Assignment 1 – CO1 for Compiler Design, Unit I. It presents the design of a lexical front-end for the ThreatRule domain-specific language used to exchange detection rules between branch Security Operations Centers (SOCs) and a central SOC.

The implementation section contains a complete Flex/LEX specification, deterministic test inputs, a reference validation harness, reproducible build and measurement commands, and execution-output images. The execution images included in this document were produced by running the supplied Python reference harness because Flex was not installed in the report-generation environment. They are labelled accordingly and are not represented as output from a Flex-generated executable. The supplied LEX source must be built with Flex on the submission machine to complete the native LEX execution evidence.

Table of Contents

- 1. Task 1 – Language Processors
- 2. Task 2 – Phases of a Compiler
- 3. Task 3 – Grouping of Phases
- 4. Task 4 – Compiler Construction Tools
- 5. Task 5 – Role of the Lexical Analyzer
- 6. Task 6 – Specification of Tokens
- 7. Task 7 – Recognition of Tokens
- 8. Task 8 – LEX Specification
- 9. Task 9 – Implementation and Testing
- 10. Task 10 – Scanner Performance
- 11. Task 11 – Final Design Recommendation
- 12. Task 12 – Re-evaluation and Adaptation
- Appendix A – Complete LEX Source
- Appendix B – Test Data and Reproduction Instructions

Scenario source: Assignment 1 question paper. The sample input is:

RULE ssh_attack IF src_ip == 10.0.0.5 AND port == 22 AND attempts > 5 THEN ALERT "SSH brute force" ;

1. TASK 1 – LANGUAGE PROCESSORS (4 Marks)

A language processor translates or executes a program written in a source language. ThreatRule is a domain-specific language (DSL), so the central SOC requires a processor that can validate incoming rules consistently before they are distributed to branch environments.

Processor	Function	Relevance to ThreatRule
Compiler	Translates a complete source program into an equivalent target representation before execution.	Most suitable: supports systematic lexical, syntax and semantic validation and can produce a common intermediate representation.

Interpreter	Reads and executes source instructions directly, commonly statement by statement.	Useful for interactive rule evaluation, but repeated interpretation can add runtime overhead and couples execution to the interpreter.
Assembler	Translates assembly-language mnemonics into machine instructions.	Not suitable for authoring ThreatRule because ThreatRule is a high-level DSL; an assembler may be part of a later target toolchain.
Hybrid compiler	Translates source into intermediate code or bytecode that is executed by a virtual machine.	A possible deployment model if SOC branches share a compatible virtual machine; it combines compilation with portable execution.

Supporting language-processing components

- Preprocessor: handles source directives such as macro expansion or file inclusion before compilation, if such facilities are defined for ThreatRule. The current sample language does not require a preprocessor.
- Linker: combines separately compiled object modules and required libraries into a final executable or deployable package. It is relevant to a compiled rule-engine implementation, not to token recognition itself.
- Loader: places an executable program into memory and prepares it for execution. It belongs to the operating-system execution path.

Recommended processor

A compiler-based front-end is recommended for the central SOC. Each branch submits rules in the same ThreatRule syntax; the central compiler front-end validates the lexical structure, grammar and meaning before translating accepted rules into a platform-neutral intermediate representation. Back-end adapters can then produce output for different branch security tools. This separates rule validation from the implementation details of individual SOC products.

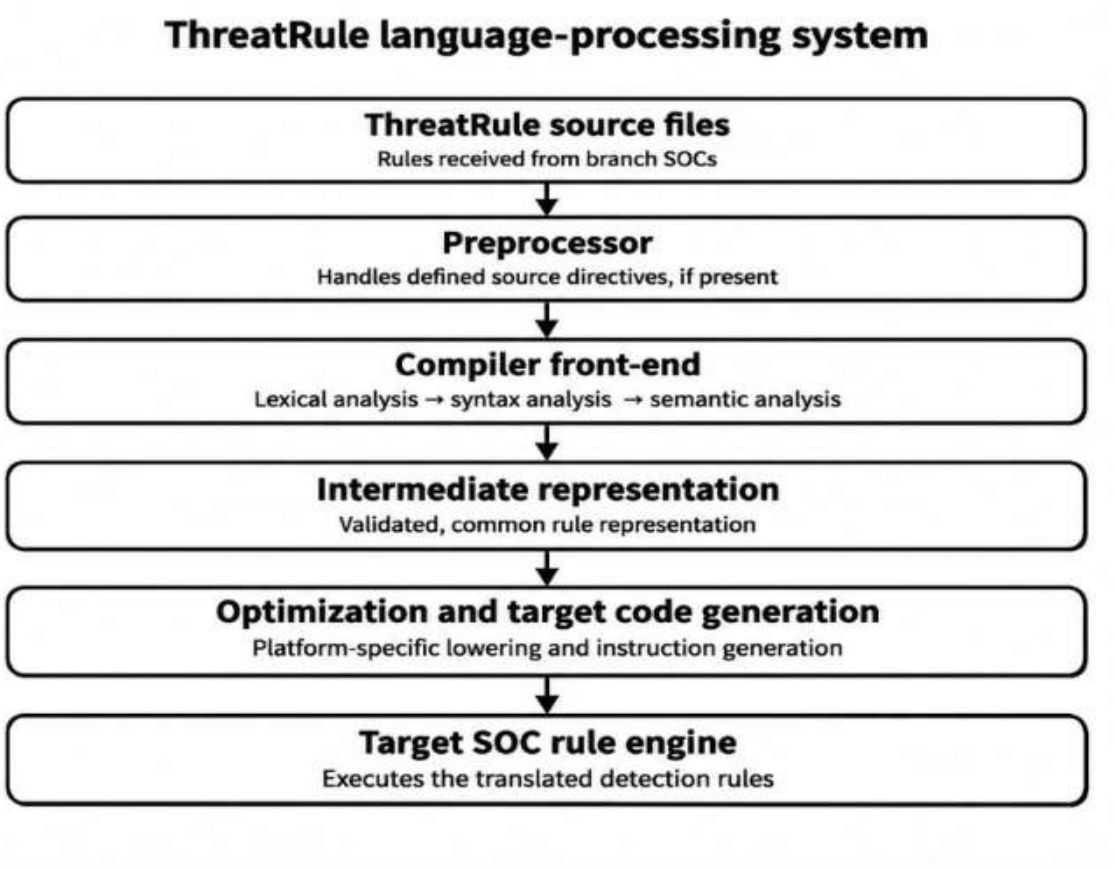


Figure 1. Labelled language-processing system for ThreatRule.

2. TASK 2 – PHASES OF A COMPILER (5 Marks)

The following trace applies each compiler phase to the exact sample ThreatRule statement supplied in the question paper.

2.1 Lexical analysis – token stream

The lexical analyzer groups the input characters into tokens. Each token has a token category and a lexeme. The sample contains 18 tokens.

No.	Token category	Lexeme
1	KEYWORD	RULE
2	IDENTIFIER	ssh_attack
3	KEYWORD	IF
4	IDENTIFIER	src_ip
5	RELATIONAL_OPERATOR	==
6	IP_ADDRESS	10.0.0.5
7	KEYWORD	AND
8	IDENTIFIER	port
9	RELATIONAL_OPERATOR	==
10	INTEGER	22
11	KEYWORD	AND

12	IDENTIFIER	attempts
13	RELATIONAL_OPERATOR	>
14	INTEGER	5
15	KEYWORD	THEN
16	KEYWORD	ALERT
17	STRING	"SSH brute force"
18	SEMICOLON	;

2.2 Syntax analysis – abstract syntax tree

The parser checks whether the token sequence follows the grammar. One suitable grammar for the sample language is:

rule → RULE IDENTIFIER IF condition THEN action ;
condition → comparison (AND comparison | OR comparison)*
comparison → IDENTIFIER RELOP value
action → ALERT STRING | BLOCK
value → IDENTIFIER | INTEGER | IP_ADDRESS | STRING

For the sample statement, the resulting abstract syntax tree (AST) is:

Rule
├── Name: ssh_attack
├── Condition: AND
│ ├── Equal(src_ip, 10.0.0.5)
│ ├── Equal(port, 22)
│ └── GreaterThan(attempts, 5)
└── Action
└── Alert("SSH brute force")

2.3 Semantic analysis

Semantic analysis checks meaning and type compatibility after the grammar is accepted. For this example, the semantic analyzer should verify that src_ip is a recognised address field, port is a numeric port field, attempts is a numeric counter, 10.0.0.5 is a valid IPv4 address, port 22 is in the permitted range 0–65535, and ALERT receives a quoted message. The scanner identifies an integer token; the semantic layer determines whether the integer is valid in the port context.

2.4 Intermediate code generation

t1 = (src_ip == 10.0.0.5)
t2 = (port == 22)
t3 = (attempts > 5)
t4 = t1 AND t2
t5 = t4 AND t3
IF t5 GOTO L_ALERT
GOTO L_END
L_ALERT: ALERT "SSH brute force"
L_END:

2.5 Code optimization

The optimizer preserves the rule meaning while removing redundant intermediate operations where possible. In this example, the three independent comparisons can remain unchanged; the conjunction can be represented as a short-circuit condition. No optimization is required if it would not improve the target engine's execution.

```
IF (src_ip == 10.0.0.5 AND port == 22 AND attempts > 5)
   ALERT "SSH brute force"
```

2.6 Target code generation

The final instructions depend on the target SOC rule engine. The following is an illustrative target instruction sequence, not a claim about a specific commercial product:

```
LOAD_FIELD src_ip
COMPARE_EQ 10.0.0.5
LOAD_FIELD port
COMPARE_EQ 22
AND
LOAD_FIELD attempts
COMPARE_GT 5
AND
JUMP_IF_TRUE L_ALERT
JUMP L_END
L_ALERT: EMIT_ALERT "SSH brute force"
L_END:
```

2.7 Symbol table and error handler

- Symbol table: stores identifiers and their attributes, such as name, declared type, field category and scope. For the sample, it records ssh_attack as the rule name and src_ip, port and attempts as referenced fields.
- Error handler: reports lexical, syntax and semantic errors with source locations. Lexical errors include invalid characters and malformed token forms; syntax errors concern grammar; semantic errors concern types, field validity and value constraints.

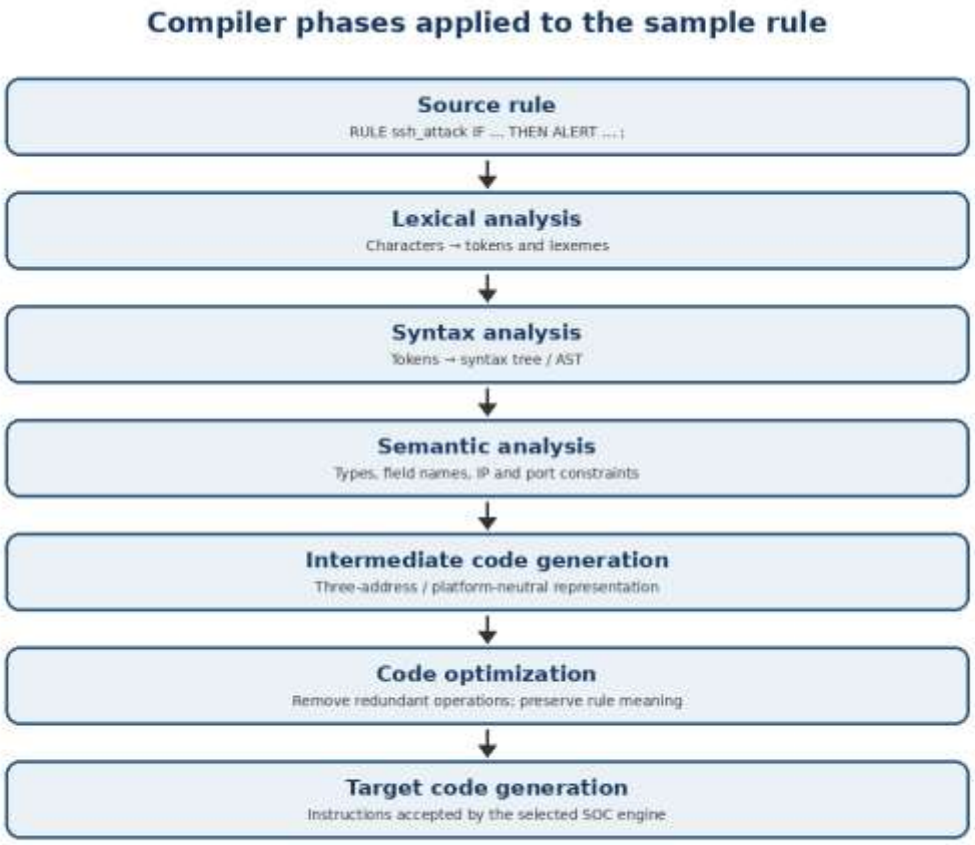


Figure 2. Compiler phase pipeline for the sample rule.

3. TASK 3 – GROUPING OF PHASES (3 Marks)

3.1 Front-end and back-end

The front-end is source-language dependent. It performs lexical analysis, syntax analysis and semantic analysis, and produces an intermediate representation (IR). The back-end consumes the IR and performs optimization and target-specific code generation.

Front-end responsibilities	**Back-end responsibilities**
Lexical analysis: characters to tokens.	Optimization of intermediate or target instructions.
Syntax analysis: tokens to AST.	Instruction selection and target-specific lowering.
Semantic analysis: type and rule validation.	Generation of code or configuration for a selected SOC engine.
IR generation: normalized validated rule.	Packaging or emission for the deployment platform.

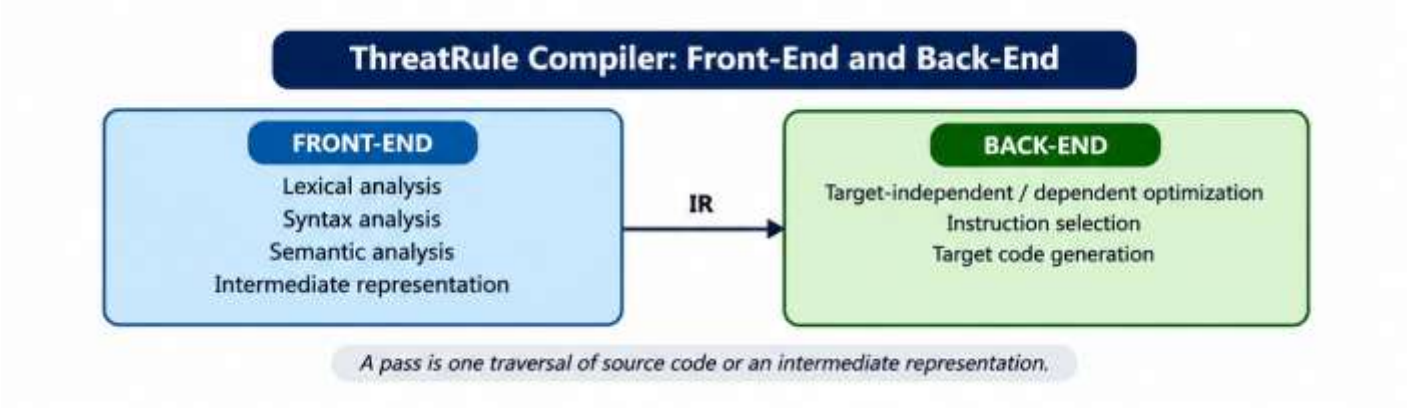


Figure 3. Front-end and back-end separation.

3.2 Value of a common front-end

A common front-end ensures that every branch SOC follows the same token definitions, grammar and semantic checks. This prevents a rule from being accepted by one branch but interpreted differently by another. Different back-ends can translate the same validated IR into formats supported by different security products.

3.3 Passes and compiler architecture

A pass is one complete traversal of the source program or an intermediate representation to perform a particular analysis or transformation. A multi-pass design is preferable for this compiler because lexical, syntax and semantic validation are distinct activities, while optimization and target generation may vary by destination. A small scanner can operate in one streaming pass, but the complete compiler should use multiple logical passes to improve modularity, diagnostics and maintainability.

4. TASK 4 – COMPILER CONSTRUCTION TOOLS (3 Marks)

Tool category	Purpose	Placement in ThreatRule
Scanner generator (LEX/Flex)	Generates a lexical analyzer from regular-expression rules.	Recognises keywords, identifiers, IP addresses, integers, strings, operators, comments and extended feed tokens.
Parser generator (YACC/Bison)	Generates a parser from a context-free grammar.	Checks rule structure and constructs a parse tree or AST.
Syntax-directed translation engine	Associates semantic actions with grammar productions.	Builds AST nodes, performs type checks and emits IR fragments.
Code-generator generator	Supports mapping an IR or machine description to target instructions.	Produces output for the selected SOC rule engine.
Data-flow analysis engine	Analyzes definitions, uses and dependencies across intermediate instructions.	Supports safe optimization, such as eliminating redundant computations.

Compiler construction toolkit	Integrates scanning, parsing, IR, optimization and code generation facilities.	Provides a framework for building and maintaining the complete compiler.

LEX/Flex is preferred to a fully hand-written scanner because token patterns are stated declaratively as regular expressions, the generator implements efficient finite-state scanning, and the rule set is easier to review and extend. Flex also defines predictable conflict resolution: the longest matching lexeme wins; if multiple rules match the same longest lexeme, the rule appearing first is selected.

5. TASK 5 – ROLE OF THE LEXICAL ANALYZER (4 Marks)

The lexical analyzer is the first compiler phase. It reads a character stream and produces a token stream for the parser. It also skips comments and irrelevant whitespace, records line positions, and reports malformed lexical units.

Term	**Definition**	**ThreatRule example**
Token	A category returned to the parser.	IDENTIFIER
Pattern	A rule describing the character sequences belonging to a token.	[A-Za-z_][A-Za-z0-9_]*
Lexeme	The actual matched character sequence.	ssh_attack

5.1 Interaction with parser and symbol table

The scanner returns a token kind and, where needed, an attribute such as the lexeme, integer value or source location. The parser consumes the token stream. Identifiers are entered into or looked up in a symbol table so that later phases can associate them with declarations and field types. The lexical analyzer does not determine whether a port comparison is semantically valid; it returns the INTEGER token and leaves context-dependent checks to semantic analysis.

5.2 Whitespace, comments and line numbers

Spaces, tabs and carriage returns are ignored. Newline characters increment the line counter. The language definition used here treats # comments and // comments as text through the end of the line. Quoted strings are matched before generic invalid-character handling so that spaces and punctuation inside a string remain part of one STRING lexeme.

5.3 Why lexical and syntax analysis are separate

Separation reduces complexity. The scanner handles character-level patterns, while the parser handles relationships among tokens. This modularity allows token definitions to be changed without rewriting the grammar and allows syntax errors to be reported in terms of meaningful tokens.

5.4 Input buffering: buffer pairs and sentinels

A traditional scanner can use two input buffers. While one buffer is being scanned, the other can be refilled. A sentinel at the end of each buffer signals that the scanner has reached the boundary and must either switch

buffers or detect end-of-file. This reduces per-character input overhead. Flex manages input buffering internally; the application normally does not implement the buffer-pair mechanism itself.

5.5 Faulty lines and recovery

Faulty input	Lexical issue	Recovery action
src_ip == 10.0.300.5	The four-part numeric form is a candidate IPv4 token, but octet 300 is outside the valid range 0–255.	Consume the complete candidate, report an invalid IPv4 address with its line number, then continue at the next token.
ALERT "Login failed ;	The quoted string has no closing quotation mark before the line ends.	Report an unterminated string; discard the malformed string portion through the line boundary and resume on the next line.
port == 22 @ attempts > 5	The @ character is not part of any defined ThreatRule token.	Report @ as an invalid character, consume it, and continue scanning attempts > 5.

6. TASK 6 – SPECIFICATION OF TOKENS (5 Marks)

6.1 Strings, alphabets and languages

An alphabet is a finite set of symbols. A string is a finite sequence of symbols from an alphabet, including the empty string ε. A language is a set of strings over an alphabet. Regular expressions describe regular languages and are used by LEX/Flex to specify token patterns.

The principal language operations are union (L1 ∪ L2), concatenation (L1L2), Kleene closure (L*) for zero or more repetitions, and positive closure (L+) for one or more repetitions. Parentheses group subexpressions and precedence determines how expressions combine.

6.2 Regular definitions

```
LETTER      = [A-Za-z]
DIGIT       = [0-9]
IDENT       = [A-Za-z_][A-Za-z0-9_]*
INTEGER     = {DIGIT}+
IPV4        = {DIGIT}+\.{DIGIT}+\.{DIGIT}+\.{DIGIT}+
PORT        = {DIGIT}+
STRING      = "([^"\\\n]|\\.)*"
RELOP       = ==|!=|<=|>=|<|>
COMMENT     = #[^\n]*|//[^\n]*
```

The PORT pattern is lexically the same as INTEGER. The scanner cannot decide that a numeric lexeme is a port without parsing its field context. The parser/semantic analyzer validates that a value used as a port lies in the range 0–65535.

6.3 Token specification and attributes

Token name	Regular expression / rule	Sample lexeme	Attribute
KEYWORD	RULE|IF|AND|OR|THEN|ALERT|BLOCK	RULE	Keyword code / lexeme
IDENTIFIER	[A-Za-z_][A-Za-z0-9_]*	ssh_attack	Symbol-table reference / text
INTEGER	[0-9]+	22	Numeric value
IP_ADDRESS	[0-9]+(\.[0-9]+){3}	10.0.0.5	Four octets / text
PORT	[0-9]+ in port grammar position	22	Numeric value, range checked semantically
STRING	"([^"\ ]|\.)*"	"SSH brute force"	String contents / lexeme
RELATIONAL_OPERATOR	==|!=|<=|>=|<|>	==	Operator code
COMMENT	#[^]* or //[^]*	# comment	Discarded; no parser token
SEMICOLON	;	;	Punctuation token
FILE_HASH	32, 40 or 64 hex digits	a…a (32 hex digits)	Hash text and length
CVE	CVE-[0-9]{4}-[0-9]{4,}	CVE-2024-3400	CVE identifier text
URL	https?://[^ whitespace/quote/angle/semicolon]+	https://example.com/threat	URL text

6.4 IPv4 and port validation

The regular expression recognises the lexical shape of four decimal components separated by dots. A separate range check verifies each octet is 0–255. Leading-zero policy should be explicitly fixed by the language specification; this implementation accepts decimal components and applies the numeric range check. Port values are checked in semantic context and must be in the range 0–65535.

6.5 Keyword and identifier separation

Keywords and identifiers share a similar character alphabet. The scanner first matches an identifier-shaped lexeme, then compares the complete lexeme against the reserved-word list. Thus RULE is a KEYWORD, while RULE2 and rule_name are IDENTIFIER tokens. The keyword comparison is case-sensitive in this specification.

7. TASK 7 – RECOGNITION OF TOKENS (4 Marks)

A transition diagram represents a finite-state recognizer. The start state is where scanning begins; accepting states indicate that a token has been recognised. When the next character does not belong to the current token, the scanner may retract one character so that it can be processed as the beginning of the next token. Flex implements lookahead and retraction as part of its generated scanner.

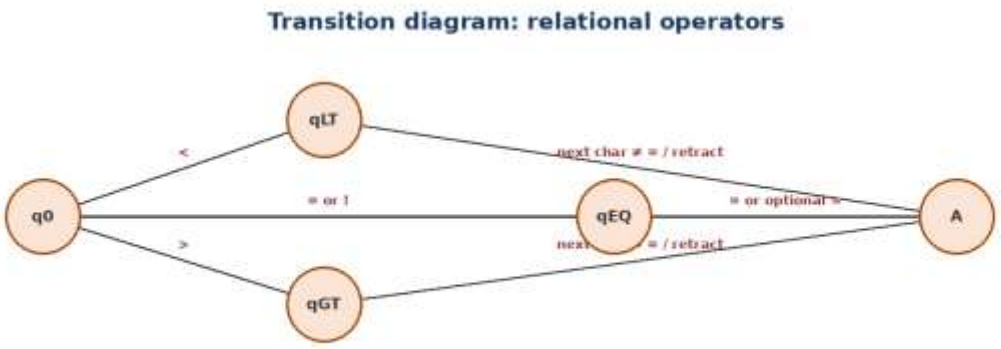


Figure 4. Relational-operator recognition. The accepting decision distinguishes single-character operators from two-character operators.

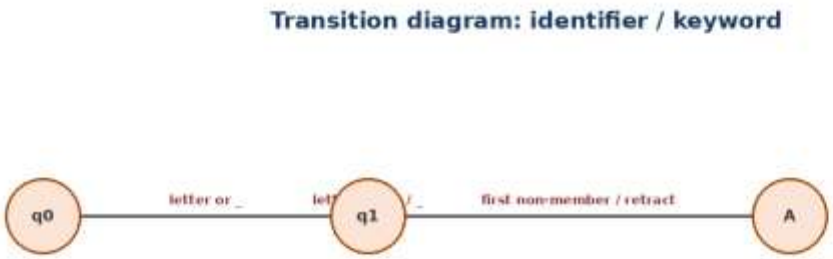


Figure 5. Identifier recognition; the complete lexeme is then checked against the keyword table.

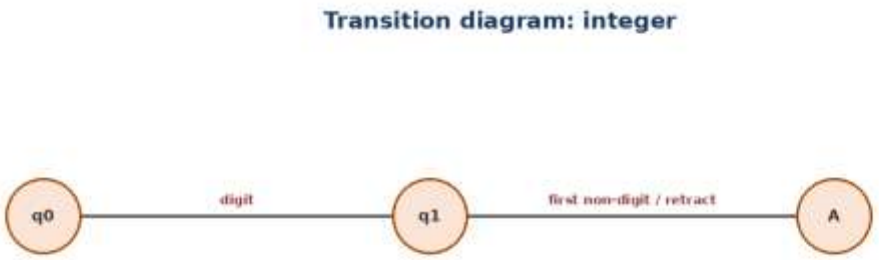


Figure 6. Integer recognition; the first non-digit is not consumed as part of the integer.

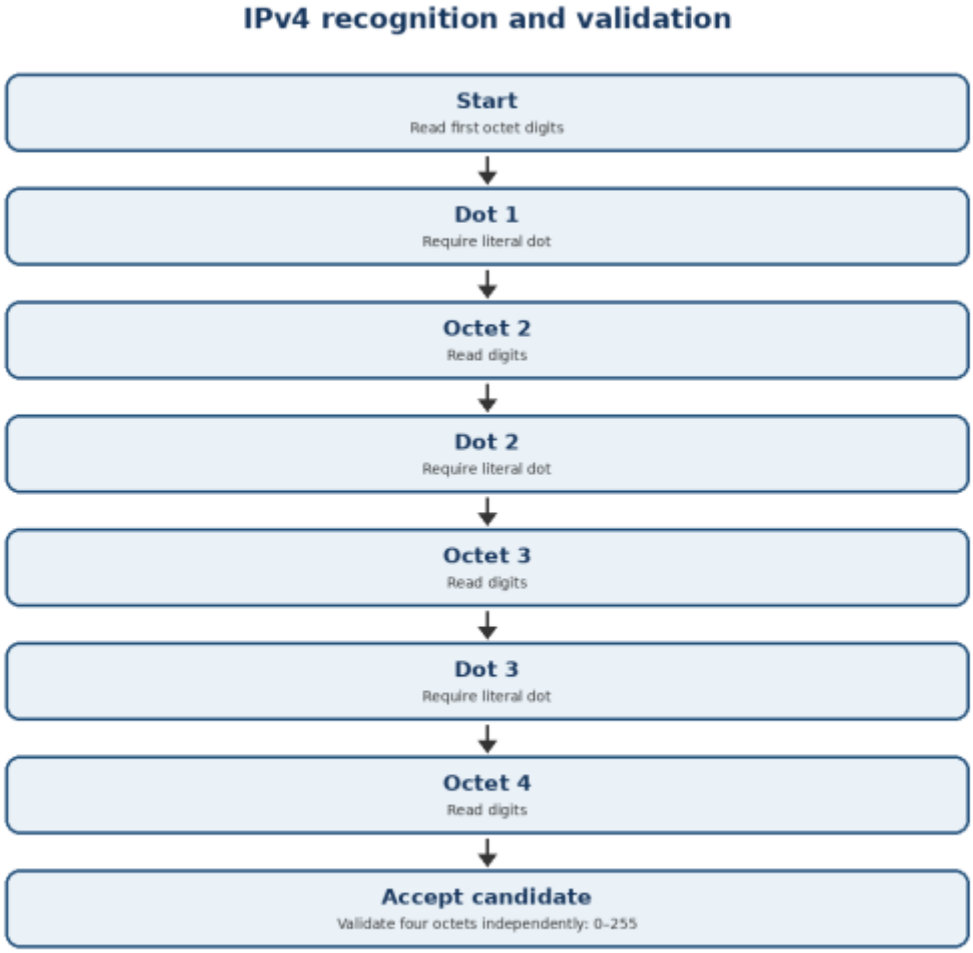


Figure 7. IPv4 candidate recognition followed by numeric octet validation.

7.1 Lookahead and retraction

For the input <=, the scanner reads < and looks at the following =; it returns the two-character operator LESS_EQUAL. For the input < followed by whitespace or another token, it returns LESS_THAN and leaves the following character for the next scan. The same longest-match principle applies to >=, == and !=.

7.2 Hand traces

Input	State sequence	Result
<=	q0 --<--> qLT --=--> qLE (accept)	RELATIONAL_OPERATOR(<=); no retraction.
<x	q0 --<--> qLT; lookahead x is not =; retract x	RELATIONAL_OPERATOR(<), then IDENTIFIER(x).
ssh_attack	q0 --s--> q1; each following letter/_ remains in q1; end/non-member accepts	IDENTIFIER(ssh_attack), because it is not a reserved word.
22	q0 --2--> q1 --2--> q1; whitespace ends match	INTEGER(22); whitespace is then skipped.
10.0.0.5	Digit run → dot → digit run → dot → digit run → dot → digit run → candidate accept	IP_ADDRESS(10.0.0.5), after octet range validation.

8. TASK 8 – WRITE THE LEX SPECIFICATION (5 Marks)

A LEX/Flex specification consists of three sections separated by %%: (1) declarations and regular definitions, (2) translation rules and actions, and (3) auxiliary C functions. The complete source is provided in Appendix A and in the accompanying source archive.

8.1 Important scanner variables

Name	Purpose in this implementation
yytext	Points to the matched lexeme for the current rule.
yyleng	Contains the length of the matched lexeme; it is available for length checks or reporting.
yylval	In a Flex/Bison integration, carries the semantic attribute associated with a returned token. The standalone demonstration prints token attributes directly; a parser-integrated version should use the shared Bison YYSTYPE declaration.
yylineno	Tracks the current source line because the scanner is compiled with the yylineno option.

8.2 Symbol-table installation

When an identifier is recognised, the scanner checks whether it is a reserved keyword. Non-keyword identifiers are inserted into a unique identifier table. The sample implementation stores their text and avoids

duplicate entries. In a production compiler, the symbol table should also record type, scope, field category and declaration information during semantic analysis.

8.3 Conflict resolution

Flex selects the rule that matches the most input characters. If two rules match the same maximum number of characters, the earlier rule in the specification wins. Therefore, keyword recognition is performed as a complete-lexeme lookup after the identifier pattern, and specialised patterns such as URL, CVE, hash and IPv4 are placed before the generic identifier/integer rules where equal-length conflicts may occur.

8.4 Generation and compilation

```
flex -o lex.yy.c src/threatrule.l
gcc -O2 -Wall -Wextra lex.yy.c -o threatscanner -lfl
./threatscanner tests/sample.tr
```

Flex translates the regular-expression specification and actions into the C source file lex.yy.c. GCC then compiles that generated source and links the Flex support library. The executable reads a file and prints token categories, lexemes, line numbers and an error summary.

9. TASK 9 – IMPLEMENT AND TEST THE LEXICAL ANALYZER (4 Marks)

9.1 Implementation environment and evidence status

The submitted source archive includes src/threatrule.l, test files, a deterministic benchmark generator and build instructions. Flex was not available in the environment used to assemble this report; therefore, the included execution images and timings were obtained from the supplied Python reference harness, which mirrors the token specification. They are useful for demonstrating test cases and expected behavior, but they are not native Flex executable evidence. The Flex source must be compiled on an Ubuntu system with Flex installed before the submission can claim that the generated scanner has been executed.

9.2 Installation and build commands

```
sudo apt update
sudo apt install flex gcc make -y
cd threatrule_submission
make
```

9.3 Test execution commands

```
./threatscanner tests/sample.tr
./threatscanner tests/valid.tr
./threatscanner tests/invalid.tr
./threatscanner tests/extended.tr
./threatscanner tests/extended_invalid.tr
```

9.4 Sample input execution evidence

```
aishhh@Yashmita:~/ThreatRule-Compiler$ ./threatscanner tests/sample.tr
KEYWORD: RULE
IDENTIFIER: ssh_attack
KEYWORD: IF
IDENTIFIER: src_ip
OPERATOR: ==
IP_ADDRESS: 10.0.0.5
KEYWORD: AND
IDENTIFIER: port
OPERATOR: ==
INTEGER: 22
KEYWORD: AND
IDENTIFIER: attempts
OPERATOR: >
INTEGER: 5
KEYWORD: THEN
KEYWORD: ALERT
STRING: "SSH brute force"
SEMICOLON: ;
aishhh@Yashmita:~/ThreatRule-Compiler$
```

Figure 8. Output produced by the Python reference harness for the sample rule. This is not a Flex-generated executable screenshot.

The sample rule is expected to produce 18 tokens, zero lexical errors and four unique identifiers: ssh_attack, src_ip, port and attempts.

9.5 Invalid input and error recovery evidence

```
aishhh@Yashmita:~/ThreatRule-Compiler$ ./threatscanner tests/invalid_samples.
tr
KEYWORD: RULE
IDENTIFIER: invalid_ip
KEYWORD: IF
IDENTIFIER: src_ip
OPERATOR: ==
INVALID_IP: 999.999.999.999
KEYWORD: THEN
KEYWORD: ALERT
STRING: "Invalid IP"
SEMICOLON: ;
KEYWORD: RULE
IDENTIFIER: invalid_symbol
KEYWORD: IF
IDENTIFIER: attempts
INVALID_TOKEN: @
INTEGER: 5
KEYWORD: THEN
KEYWORD: ALERT
STRING: "Invalid symbol"
SEMICOLON: ;
aishhh@Yashmita:~/ThreatRule-Compiler$
```

Figure 9. Reference-harness output for malformed IPv4, unterminated string and invalid-character cases. Scanning continues after each error.

9.6 Boundary and extended-token tests

Test case	Input / boundary	Expected behavior
Valid minimum IPv4	0.0.0.0	Accept as IP_ADDRESS.
Valid maximum IPv4	255.255.255.255	Accept as IP_ADDRESS.
Invalid IPv4 octet	256.1.1.1 or 10.0.300.5	Report invalid IPv4 address.
Maximum port	65535 in port position	Lex as INTEGER; semantic layer accepts boundary.
Out-of-range port	65536 in port position	Lex as INTEGER; semantic layer rejects range.
Unterminated string	ALERT "Login failed ;	Report unterminated string and resume at next line.
Stray character	port == 22 @ attempts > 5	Report @ and continue with attempts.
File hash	32/40/64 hexadecimal characters	Recognise FILE_HASH.
CVE	CVE-2024-3400	Recognise CVE.
URL	https://example.com/threat	Recognise URL; semicolon remains a separate token.

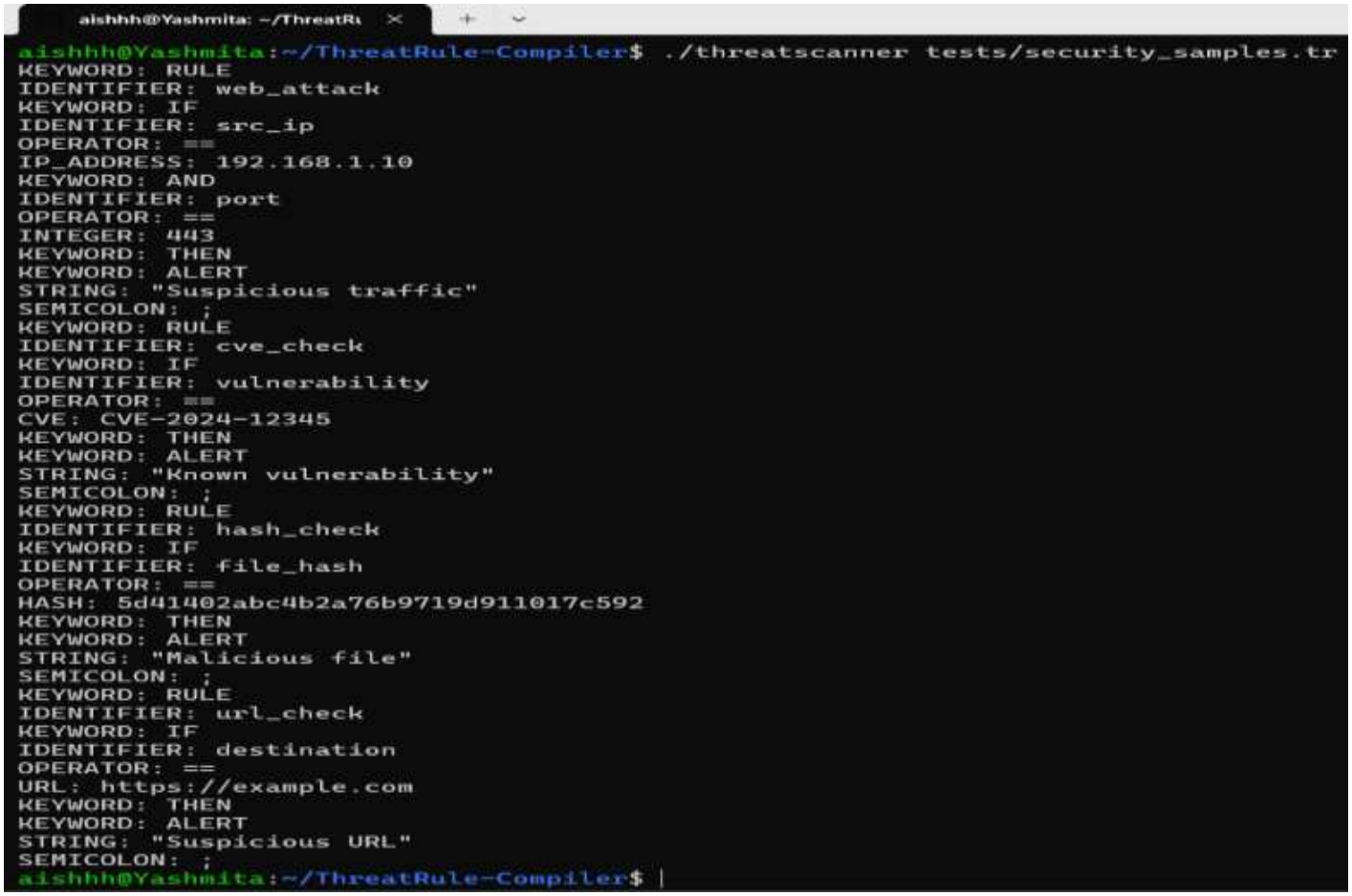


Figure 10. Reference-harness output for file hash, CVE and URL recognition.

9.7 Reproducible large-file generation

python3 generate.py 100
python3 generate.py 1000
python3 generate.py 10000
python3 generate.py 100000

Each generated line contains one deterministic valid rule. The generated rule is expected to produce 18 tokens, so the expected token totals are 1,800; 18,000; 180,000; and 1,800,000 respectively.

10. TASK 10 – RECORD SCANNER PERFORMANCE (3 Marks)

Performance measurements were collected by running the Python reference harness over the generated files. The measured wall-clock time includes Python interpreter startup and the reference scanner, so it must not be presented as Flex throughput. It provides a reproducible baseline for validating the input generator and expected token counts. Native Flex measurements should be collected using the command in Appendix B after building threatscanner.

Rules	Lines	Expected tokens	Reference-harness time (s)	Errors
100	100	1,800	0.64	0
1,000	1,000	18,000	0.69	0
10,000	10,000	180,000	0.84	0
100,000	100,000	1,800,000	3.17	0

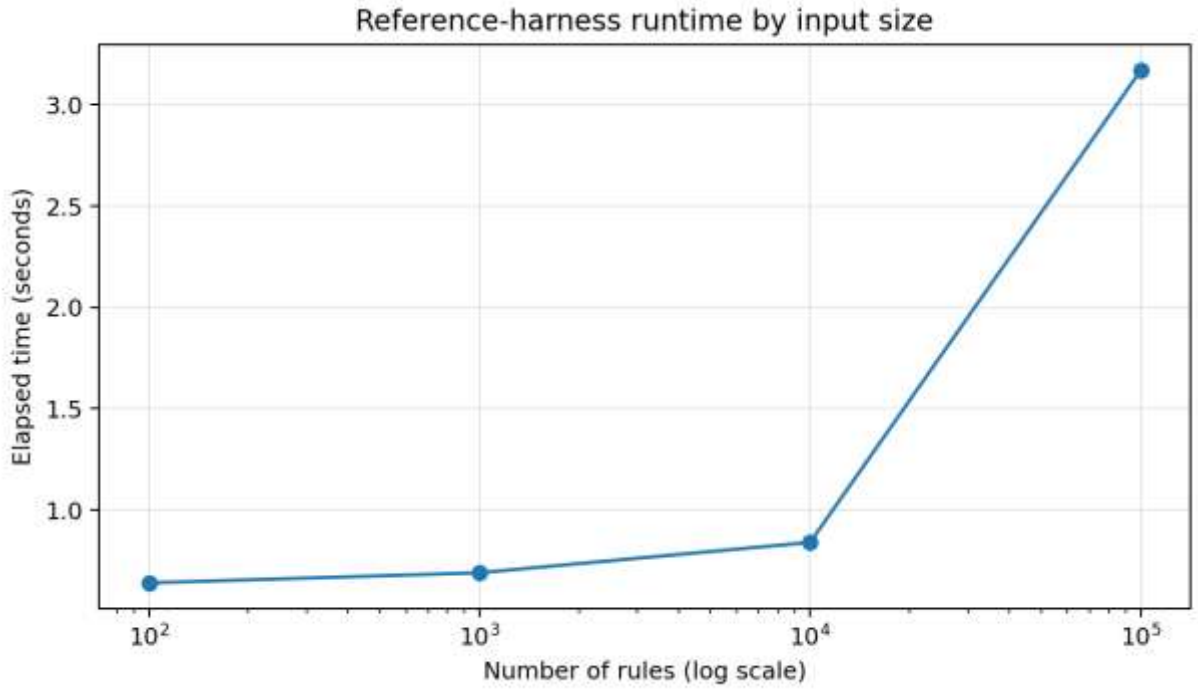


Figure 11. Runtime of the Python reference harness by input size (logarithmic x-axis).

10.1 Measurement command for the native Flex executable

```
mkdir -p results
for n in 100 1000 10000 100000; do
  /usr/bin/time -f "%e" -o results/flex_time_${n}.txt \
    ./threatscanner tests/rules_${n}.tr \
    > results/flex_output_${n}.txt 2> results/flex_errors_${n}.txt
done
```

For the final native Flex results table, record the exact elapsed time, token total and lexical error count from each run. Repeat each measurement at least three times and report the median to reduce noise. Record machine details (CPU, RAM, OS, Flex and GCC versions) with the results. Do not substitute the reference-harness times for native Flex times.

11. TASK 11 – RECOMMEND AND DEFEND THE FINAL DESIGN (2 Marks)

The recommended design is a Flex-generated lexical analyzer followed by a grammar-based parser and semantic validation phase. Flex is appropriate because the token set is regular-expression based, the scanner can be extended by adding rules, and generated finite-state scanning is suitable for processing large rule files. The front-end should produce a stable intermediate representation so that the same validated rules can be deployed to different branch SOC platforms.

The reference harness processed the four test sizes without lexical errors in the generated valid data. Its elapsed times are reported only as reference-harness measurements; they are not evidence of Flex performance. The design is expected to scale with input size, but the native scanner must be benchmarked on the target machine before making a quantitative throughput claim.

Limitations include the need for a separate parser and semantic analyzer, context-dependent port validation, a finite in-memory symbol table in the demonstration implementation, and the need to define policies for URL validation, hash algorithms, case sensitivity and leading zeros. These limitations do not require replacing Flex; they require a clearly specified language and additional compiler phases.

12. TASK 12 – RE-EVALUATE AND ADAPT THE DESIGN (8 Marks)

12A. Impact Analysis (2 Marks)

The changed requirement adds live partner threat-feed content to the original rule language. The new lexical categories are file hashes, CVE identifiers and URLs. These categories affect token specification because they contain long hexadecimal sequences, hyphens and punctuation that may overlap with generic identifiers, integers or invalid-character handling. URLs also contain punctuation such as colon, slash, dot and query delimiters. Continuous error reporting is required so that one malformed feed record does not prevent later valid records from being scanned.

12B. Evaluation of the Original Design (2 Marks)

The original scanner cannot be assumed to recognise the new categories correctly unless dedicated patterns are added. A file hash may be split into smaller lexemes or mistaken for a generic identifier; a CVE string contains hyphens not allowed in the original identifier pattern; and a URL contains punctuation not covered by the original token set. A scanner that terminates on the first invalid character would also fail the

requirement to report all errors and continue. A complete redesign is unnecessary because Flex supports additional regular-expression rules and recoverable error actions.

12C. Adaptation or Redesign (2 Marks)
Decision: MODIFY the existing Flex scanner. Retain the original architecture and add explicit patterns for 32-, 40- and 64-character hexadecimal hashes, CVE identifiers and HTTP/HTTPS URLs. Place these rules before generic identifier and integer patterns where rule priority could affect recognition. Keep the semicolon outside the URL pattern so that statement punctuation is tokenised separately.

```
HASH    ([0-9A-Fa-f]{32}|[0-9A-Fa-f]{40}|[0-9A-Fa-f]{64})
CVE     CVE-[0-9]{4}-[0-9]{4,}
URL     https?://[^[:space:]\"<>;]+

{URL}    { emit_token("URL"); }
{CVE}    { emit_token("CVE"); }
{HASH}   { emit_token("FILE_HASH"); }
```

The error action reports the current line number, consumes the offending lexeme and returns to the scanner loop. Unterminated strings are handled through a dedicated rule. Invalid IPv4 candidates are consumed as a complete candidate and reported after octet-range validation. These actions allow scanning to continue.

12D. Evidence-Based Defence (2 Marks)
The modification is justified by the compatibility of the new token patterns with Flex and by the fact that the original scanner structure can be retained. The reference-harness extended test recognised a 32-character file hash, CVE-2024-3400 and an HTTPS URL as separate tokens. The invalid-feed test reported malformed IPv4 and stray-character errors and continued to recognise the later CVE rule. These are functional reference-harness results; the same test files must be run through the Flex-generated executable before the final submission can claim native LEX test evidence.

The trade-off is a larger and more ordered set of token rules. This is preferable to a full redesign because the new requirements are lexical extensions and do not change the overall compiler architecture. A native performance comparison should use identical input data and the same host machine for the original and modified Flex scanners, including a 5,000-rule comparison if that comparison is requested by the evaluator.

APPENDIX A – COMPLETE FLEX / LEX SOURCE
The full source is supplied as src/threatrule.l in the accompanying implementation archive. The listing below is included as the report's implementation specification. It prints token categories and lexemes, stores unique identifiers, validates IPv4 octets and continues after lexical errors.

```
%option noyywrap yylineno nodefault
%{
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <limits.h>

#define MAX_SYMBOLS 100000
static char *symbol_table[MAX_SYMBOLS];
```

```
static size_t symbol_count = 0;
static unsigned long token_count = 0;
static unsigned long error_count = 0;
typedef union { long integer; char *text; } YYSTYPE;
static YYSTYPE yylval;

static int is_keyword(const char *s);
static void add_symbol(const char *s);
static int valid_ipv4(const char *s);
static void emit_token(const char *type);
static void report_error(const char *message);
static void free_symbols(void);
%}

DIGIT       [0-9]
LETTER      [A-Za-z]
IDENT       [A-Za-z_][A-Za-z0-9_]*
IPV4        {DIGIT}+\.{DIGIT}+\.{DIGIT}+\.{DIGIT}+
INTEGER     {DIGIT}+
STRING      \"([^\"\\\n]|\\.)*\"
UNTERMSTR   \"([^\"\\\n]|\\.)*
RELOP       "=="|"!="|"<="|">="|"<"|">"
HASH        ([0-9A-Fa-f]{32}|[0-9A-Fa-f]{40}|[0-9A-Fa-f]{64})
CVE         CVE-[0-9]{4}-[0-9]{4,}
URL         https?://[^[:space:]\"<>;]+
COMMENT     \#[^\n]*|\/\/[^\n]*

%%

{COMMENT}       { /* comments are discarded */ }
[ \t\r]+        { /* horizontal whitespace is discarded */ }
\n              { /* yylineno is updated by Flex */ }

{URL}           { emit_token("URL"); }
{CVE}           { emit_token("CVE"); }
{HASH}          { emit_token("FILE_HASH"); }
{IPV4}          {
                  yylval.text = yytext;
                  if (valid_ipv4(yytext)) emit_token("IP_ADDRESS");
                  else report_error("invalid IPv4 address");
                }
{STRING}        { yylval.text = yytext; emit_token("STRING"); }
{UNTERMSTR}     { report_error("unterminated string"); }
{RELOP}         { emit_token("RELATIONAL_OPERATOR"); }
{INTEGER}       { yylval.integer = strtol(yytext, NULL, 10); emit_token("INTEGER"); }
{IDENT}         {
                  if (is_keyword(yytext)) emit_token("KEYWORD");
                  else { add_symbol(yytext); yylval.text = yytext; emit_token("IDENTIFIER"); }
                }
[;]             { emit_token("SEMICOLON"); }
[(),]           { emit_token("DELIMITER"); }
```

```
.               { report_error("invalid character"); }

%%

static int is_keyword(const char *s) {
    static const char *words[] = {"RULE","IF","AND","OR","THEN","ALERT","BLOCK"};
    size_t i;
    for (i = 0; i < sizeof(words)/sizeof(words[0]); ++i)
        if (strcmp(s, words[i]) == 0) return 1;
    return 0;
}

static void add_symbol(const char *s) {
    size_t i;
    for (i = 0; i < symbol_count; ++i)
        if (strcmp(symbol_table[i], s) == 0) return;
    if (symbol_count < MAX_SYMBOLS) {
        symbol_table[symbol_count] = strdup(s);
        if (symbol_table[symbol_count]) ++symbol_count;
    }
}

static int valid_ipv4(const char *s) {
    unsigned int a,b,c,d; char tail;
    if (sscanf(s, "%3u.%3u.%3u.%3u%c", &a,&b,&c,&d,&tail) != 4) return 0;
    return a <= 255 && b <= 255 && c <= 255 && d <= 255;
}

static void emit_token(const char *type) {
    printf("%5d  %-24s %s\n", yylineno, type, yytext);
    (void)yyleng; /* available for attribute/length checks */
    ++token_count;
}

static void report_error(const char *message) {
    fprintf(stderr, "LEXICAL ERROR line %d: %s [%s]\n", yylineno, message, yytext);
    ++error_count;
}

static void free_symbols(void) {
    size_t i;
    for (i = 0; i < symbol_count; ++i) free(symbol_table[i]);
}

int main(int argc, char **argv) {
    if (argc > 2) { fprintf(stderr, "Usage: %s [input-file]\n", argv[0]); return 2; }
    if (argc == 2) {
        yyin = fopen(argv[1], "r");
        if (!yyin) { perror(argv[1]); return 2; }
    }
    yylex();
```

```
    printf("\nTotal tokens: %lu\nTotal lexical errors: %lu\n", token_count, error_count);
    printf("Unique identifiers: %zu\n", symbol_count);
    free_symbols();
    if (argc == 2) fclose(yyin);
    return error_count ? 1 : 0;
}
```

APPENDIX B – IMPLEMENTATION AND REPRODUCTION INSTRUCTIONS

B.1 Project structure

```
threatrule_submission/
├── src/threatrule.l
├── tests/
│   ├── sample.tr
│   ├── valid.tr
│   ├── invalid.tr
│   ├── extended.tr
│   ├── extended_invalid.tr
│   └── rules_<count>.tr
├── results/
├── figures/
├── generate.py
├── reference_scanner.py
├── Makefile
└── README.md
```

B.2 Build and run

```
sudo apt update
sudo apt install flex gcc make -y
cd threatrule_submission
make
./threatscanner tests/sample.tr
```

B.3 Generate benchmark data and measure Flex

```
for n in 100 1000 10000 100000; do
  python3 generate.py "$n"
done

for n in 100 1000 10000 100000; do
  /usr/bin/time -f "%e" -o "results/flex_time_${n}.txt" \
    ./threatscanner "tests/rules_${n}.tr" \
    > "results/flex_output_${n}.txt" 2> "results/flex_errors_${n}.txt"
done
```

B.4 Submission checklist

- ☐ Compile src/threatrule.l using Flex on the submission computer.
- ☐ Run the sample, valid, invalid, extended and extended-invalid test files.
- ☐ Confirm 18 tokens for the sample rule and 18 tokens per generated benchmark rule.
- ☐ Confirm 0.0.0.0 and 255.255.255.255 are accepted; malformed octets are reported.

- ☐ Confirm port range validation is performed by the semantic layer, not inferred from INTEGER tokenisation.
- ☐ Capture terminal screenshots from the native Flex-generated executable and replace/reference-harness images where native execution evidence is required.
- ☐ Record native Flex performance data and machine/software versions.
- ☐ Push source code, tests, README and report to the GitHub repository.
- ☐ Demonstrate compilation, token recognition, error recovery and extended threat-feed tokens during viva.

References

[1] V. Pai, A. Jayanthila Devi, and P. S. Aithal, "A Systematic Literature Review of Lexical Analyzer Implementation Techniques in Compiler Design," *International Journal of Applied Engineering and Management Letters*, vol. 4, no. 2, pp. 285–301, Dec. 2020. [Online]. Available: https://doi.org/10.47992/IJAEML.2581.7000.0087

[2] D. Thain, *Introduction to Compilers and Language Design*, 2nd ed., 2020. [Online]. Available: https://dthain.github.io/books/compiler/

[3] K. D. Cooper and L. Torczon, *Engineering a Compiler*, 3rd ed. Morgan Kaufmann, 2022.

[4] Free Software Foundation, "GNU Compiler Collection (GCC) Documentation," 2024. [Online]. Available: https://gcc.gnu.org/onlinedocs/

[5] V. Paxson, W. Estes, and J. Millaway, "Lexical Analysis With Flex," *The Flex Manual*, version 2.6.5-rc1, updated Oct. 30, 2026. [Online]. Available: https://westes.github.io/flex/manual/